ASSIGNMENT 5 - CO5

PART A - DESIGN

TASK A5 - DESIGN THE CLOUD DEPLOYMENT AND ORCHESTRATION

4 Marks | Bloom's Level: K5 - Evaluate

Case Study and Design Objective

The question describes a university result portal used by a few hundred students on ordinary days, with 150,000 students logging in within a two-hour result-day window. The current portal runs on one physical Linux server and crashes during the peak. The university has an account with a public cloud provider. The design must compare IaaS, PaaS and SaaS, propose a cloud arrangement with a load balancer and managed database, and explain how Kubernetes supports scaling and recovery.

The stated two-hour average is 150,000 ÷ 7,200 = 20.83 logins per second. This is only an average login-arrival rate, not the peak rate or the total HTTP/API/database request rate. Each login may trigger multiple requests. Production capacity must therefore be set from concurrency and load-test measurements, not this average alone.

1. A5.1 - Cloud Set-up (2 Marks)

1.1 IaaS, PaaS and SaaS comparison

Model	What the provider manages	What the university manages	Fit for this portal
IaaS Infrastructure as a Service	Physical servers, storage, networking and virtualisation layer.	Guest OS, patches, runtime, application, data, firewall configuration and scaling logic.	Flexible if the team needs full OS control, but it has more system-administration effort.
PaaS Platform as a Service	Infrastructure plus OS/runtime/platform services; some offerings include managed application hosting or managed Kubernetes.	Application code, data, configuration, identity/access, secrets and application-level scaling settings.	Good fit when the university wants to deploy its own portal without managing every host component.
SaaS Software as a Service	A complete provider-hosted application and its underlying platform/infrastructure.	User access, limited configuration, data governance and integration choices.	Least suitable for a custom university result portal unless an existing SaaS student-information/results product meets the requirements.

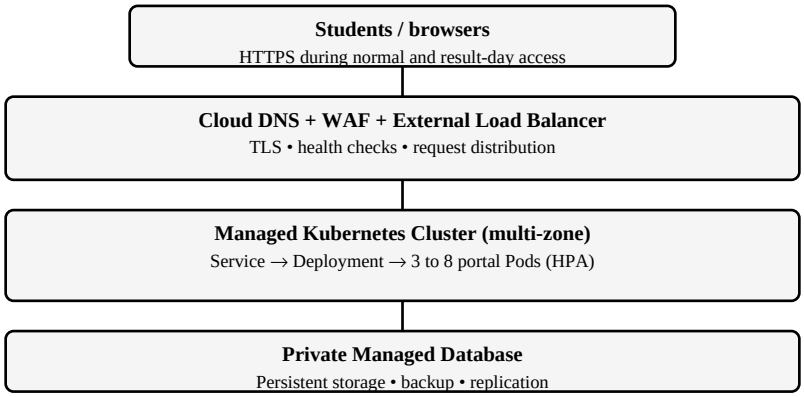
The three models differ mainly in how responsibility is divided. IaaS provides basic compute, storage and network resources; PaaS lets the customer deploy applications on a provider-managed platform; SaaS gives the customer access to a complete provider application. [1]

1.2 Recommended cloud model

Use a managed Kubernetes service for the stateless Python portal tier, combined with a cloud load balancer and a managed relational database. This is a managed-platform approach: the provider operates the Kubernetes control plane and the university manages application containers, deployment configuration, resource policies, data and access controls. If the university needs custom host drivers or OS-level agents, IaaS virtual machines can be used as Kubernetes worker nodes, but the control plane and database should still be managed where practical.

2. Proposed Cloud Architecture

The proposed request path is: student browser → DNS/WAF → external load balancer → Kubernetes Service → ready portal Pods → private managed database. Deploy the Kubernetes worker pool across at least two availability zones (preferably three where budget and regional availability permit). Keep the database on private networking and allow connections only from the portal workload identity or approved cluster subnet.



Component	Design and responsibility
DNS and WAF	Resolve the public portal hostname; filter common web threats and abusive traffic before it reaches the application.
External load balancer	Terminate TLS or forward to the cluster ingress/service, perform health checks and distribute incoming HTTPS traffic.
Managed Kubernetes control plane	Provider operates the API/control-plane infrastructure; the university configures cluster access, namespaces, policies and deployments.
Worker nodes	Run portal Pods. Use a managed node pool with node autoscaling; distribute nodes and replicas across failure zones.
Portal Deployment and Service	Deployment maintains the desired Pod replicas; Service provides a stable virtual endpoint and routes to matching ready Pods.
Managed database	Private relational database with automated backups, point-in-time recovery and high-availability/replica options according to the selected service tier.
Monitoring and secrets	Collect application/cluster/database metrics and logs; store database credentials and TLS material in a secret manager rather than in images or plain manifests.

The database is not exposed directly to the public internet. Use connection pooling, prepared queries, suitable indexes and a tested connection limit so that scaling application Pods does not overwhelm the database. Database capacity, replicas and storage must be tested separately; increasing web replicas does not automatically increase database throughput.

2.1 Why this design fits result day

The application tier is largely stateless and can be replicated horizontally. If one portal Pod fails, the Deployment controller can replace it and the Service can stop routing traffic to a Pod that is not ready. More replicas can be added as demand rises. The managed database retains persistent result records outside the lifecycle of individual application Pods. Multi-zone worker placement and database high availability reduce reliance on a single physical machine.

3. A5.2 - Kubernetes Orchestration (2 Marks)

Kubernetes object	Meaning	Role in the portal
Pod	Smallest deployable unit; one or more closely related containers share network and storage context.	Runs one portal application container, with any tightly coupled sidecar only when needed.
Deployment	Declares the desired application version and replica state; manages ReplicaSets and rolling updates.	Maintains the portal Pod count, replaces failed Pods and performs controlled application releases.
Service	Stable network endpoint that selects Pods by labels and routes traffic to available endpoints.	Gives the load balancer/cluster a stable destination while Pod IPs change.
Replicas	Multiple instances of the same Pod template.	Spread requests and continue serving when an individual Pod becomes unavailable.
Resource requests	CPU and memory needed for scheduling and capacity planning.	Kubernetes scheduler uses the request when choosing a node; the request also provides the CPU baseline for HPA utilisation.
Resource limits	Maximum permitted resource amount for a container.	CPU limit can throttle use; memory over limit can result in OOM termination. Values must be load-tested.
Horizontal Pod Autoscaler	Controller that changes a scalable workload's desired replica count based on observed metrics.	Adds portal Pods when CPU/custom load rises and reduces them when demand falls, within configured minimum and maximum.

A Pod is the unit Kubernetes schedules and manages; a Deployment declares the desired number and version of replicas and controls replacement/rolling updates. A Service provides a stable address and selects the Pods that should receive traffic. [2][3][4]

3.1 Requests, limits and scheduling

A resource request tells the scheduler how much CPU and memory to reserve for placement decisions. A limit caps the container's consumption. CPU requests are used to calculate CPU utilisation percentages for HPA; therefore, CPU requests must be set when using CPU-utilisation-based autoscaling. Too-low requests can cause overpacking and inflated utilisation; too-high requests can leave capacity unusable and prevent new Pods from scheduling. [5]

4. How Scaling and Recovery Work

4.1 Horizontal Pod Autoscaler

The HPA controller periodically reads metrics for Pods in a target Deployment and calculates the desired replica count. For example, it can target average CPU use at 65% of the CPU request. If average utilisation stays above the target, HPA increases the desired replicas, up to the maximum. When usage falls, it can reduce replicas, subject to stabilisation settings. CPU/resource metrics normally require a working resource metrics API such as metrics-server; external/custom request-rate metrics need a compatible metrics adapter. [6]

HPA scales Pods, not the underlying cloud virtual machines. If new Pods cannot be scheduled because worker nodes are full, configure the managed node pool's cluster autoscaler or node auto-provisioning to add worker nodes. When demand falls, node autoscaling can remove unneeded nodes after Pods have been moved safely. The HPA and node autoscaler therefore operate at different levels.

Control	Illustrative setting in supplied manifest	Reason
Minimum replicas	3	Keep a small, multi-instance baseline for normal traffic and reduce single-Pod dependence.
Maximum replicas	8	Provide an upper bound for the worked example; final value must be proven by load testing and database capacity.
CPU target	65% of requested CPU	Start scaling before containers remain continuously saturated.
Scale-down stabilisation	300 seconds	Avoid rapid scale-in when traffic briefly drops between bursts.
Deployment update	Rolling update; maxUnavailable 0, maxSurge 1	Introduce new Pods before removing old ones where capacity allows.
Pod disruption budget	At least 2 available Pods	Reduce voluntary maintenance disruption; does not guarantee recovery from all failures.

4.2 Recovery sequence

Failure or event	Kubernetes / cloud response	What must also be configured
Container exits	Kubelet restarts a container according to the Pod restart policy; Deployment/ReplicaSet maintains desired Pod count if a Pod is lost.	Health probes, logs, alerting and an application that can restart safely.
Pod becomes unhealthy	Readiness probe marks it not ready; Service endpoints stop sending new traffic to it. Liveness can trigger restart after repeated failures.	Correct health endpoints and suitable probe thresholds/timeouts.
Worker node fails	Pods on the failed node become unavailable; controllers can create replacement Pods on healthy nodes if resources exist.	Multiple nodes/zones, spare capacity, persistent-data strategy and node autoscaling.
Traffic rises	HPA increases desired replicas; Service/load balancer sends requests to ready Pods.	Metrics API, sufficient node capacity, pre-scaling for known events and database connection limits.
Database failure	Application replicas remain available but cannot serve database-dependent requests normally.	Managed database HA/failover, backups, restore tests, connection retry/backoff and clear degraded-mode handling.

For a scheduled result release, pre-scale the portal before the expected login window. HPA is reactive: metric collection, scheduling, image pull, container start, readiness checks and possibly node provisioning all take time. Pre-scaling warm capacity reduces the risk of scaling lag at the beginning of the event.

5. Alternative Concept - Fixed VMs Sized for Result Day

An alternative is to keep enough fixed virtual machines running all year to handle the estimated peak. This can provide immediate capacity because the VMs are already booted, but it reserves resources during ordinary days when the portal serves only a few hundred students. The fixed design also remains vulnerable if peak demand exceeds the original estimate or if one host/database/network component fails.

Evaluation	Fixed result-day VMs	Kubernetes with HPA
Result-day response	Pre-running instances avoid Pod/VM scale-up delay; response can be predictable if all tiers are sized and healthy.	Scales Pod count in response to metrics, but scheduling, image pulls and node provisioning add delay. Pre-scale before a known event and retain minimum warm replicas.
Other 364 days	All peak-sized VM capacity remains allocated, although normal usage is much lower.	Portal replicas can return to the minimum; node autoscaling can reduce excess worker nodes. Managed database and load balancer remain provisioned.
Cost pattern	High and relatively constant compute cost; pay for peak capacity even when idle.	Variable application compute; lower idle capacity but incurs managed Kubernetes, node, load balancer, database and monitoring costs.
Operational work	Simple fixed count, but patching, health checks, failover and capacity estimates remain manual/automated VM operations.	More moving parts and Kubernetes skills; controllers, HPA and managed node pools automate replica and node changes.

5.1 Worked idle-resource calculation

The assignment does not supply instance sizes, cloud prices or measured normal/peak load. The following is an illustrative model to show the comparison method; it is not a production sizing result.

Planning input	Illustrative value
Ordinary-day portal baseline	3 application instances
Result-day application capacity	8 application instances
Equivalent capacity per instance	1 vCPU request and 2 GiB RAM
Elevated result-day window	4 hours (two-hour login window plus pre-scale and cool-down)
Year length	365 days = 8,760 hours

Fixed VMs: eight instances remain allocated continuously. Relative to a three-instance ordinary-day baseline, five extra instances are idle or underused on the other 364 days.

5 instances × 24 hours × 364 days = 43,680 extra instance-hours.

Equivalent reserve over those 364 days: **5 vCPU × 8,736 hours = 43,680 vCPU-hours** and **10 GiB × 8,736 hours = 87,360 GiB-hours** of reserved memory capacity.

5.2 Annual application compute comparison

Fixed VM model: **8 × 8,760 = 70,080 instance-hours/year.** HPA model: **(3 × 8,760) + (5 × 4) = 26,300 replica-hours/year.**

Illustrative reduction in application instance-hours: (70,080 – 26,300) ÷ 70,080 × 100 = **62.47%**. This is a comparison of application capacity-hours only. It does not mean the total cloud bill falls by 62.47%, because control-plane, node, database, storage, load balancer, network and observability charges remain.

6. Cost Evaluation, Risks and Final Recommendation

6.1 Cost model

For a provider-neutral estimate, calculate cost from each service rather than inventing a single cloud price. The fixed-VM application compute estimate is eight VM instances multiplied by 8,760 hours and the selected hourly price. The autoscaled application compute uses the replica-hours from the worksheet, multiplied by equivalent capacity cost, then adds node headroom and managed service charges.

Cost component	Fixed VM approach	Managed Kubernetes + HPA
Application compute	8 app VMs × 8,760 h × VM hourly rate.	Autoscaled Pod/node capacity-hours × node rate; include minimum node reserve.
Kubernetes control plane	Not required if VMs are managed individually; orchestration/management may be separate.	Managed cluster/control-plane fee, if charged by provider.
Load balancer	Required for distributing traffic across fixed VMs.	Required for external traffic to managed cluster/Service.
Database	Managed database or dedicated DB VM, sized for peak.	Same managed database cost; scale/HA tier is independent of HPA.
Storage / backup	VM disks, database storage, snapshots and backup retention.	Persistent database storage, backup retention and any shared storage.
Network / operations	Traffic egress, monitoring, logs, patching and staff effort.	Traffic egress, metrics/logs, cluster operations, security and staff effort.

The exact cost cannot be calculated from the assignment alone because no provider, region, instance family, database tier, storage size, egress volume or currency is given. Use the chosen provider's pricing calculator and enter the actual workload. The main conclusion is directional: fixed capacity buys immediate peak readiness at the cost of idle resources; autoscaling can reduce application compute outside the peak but introduces management complexity and scale-up delay.

6.2 Risks and controls

Risk	Control
HPA reacts too late for the sudden login spike.	Pre-scale before the known result-release time; maintain minimum replicas; test image pull and readiness times.
Worker nodes are full, so additional Pods stay Pending.	Enable managed node autoscaling and keep capacity headroom across zones.
CPU-based HPA misses non-CPU bottlenecks.	Monitor request rate, latency, queue depth and database connections; add custom/external metrics where justified.
Application replicas overload the database.	Use connection pooling, cap per-Pod connections, tune query/indexes and scale the managed database separately.
Pods are lost during host/zone failure.	Spread replicas across nodes/zones, use readiness probes and a PodDisruptionBudget, and test failover.
Secrets or data are exposed.	Use a secret manager, private DB networking, workload identity, least-privilege access and encryption.
Autoscaling causes unexpected cost.	Set max replicas and node-pool limits, budgets/alerts and event-based cost monitoring.

6.3 Final recommendation

Adopt a managed Kubernetes service for the stateless Python portal, with a cloud load balancer, at least three baseline replicas, an HPA limit established by load testing, and managed node autoscaling. Deploy worker nodes across availability zones, use readiness/liveness probes and rolling updates, and keep the managed database private with high availability, backups and tested restore. Pre-scale ahead of the result-day window rather than relying only on reactive HPA.

This design fits the seasonal workload better than keeping all peak-sized VMs running for 365 days. The worked model shows five additional instances reserved for 364 normal days in the fixed design, while the HPA model activates them around the event. The actual economic saving must be calculated using provider rates and measured load; it is not a guaranteed percentage saving.

7. Conclusion

IaaS, PaaS and SaaS differ in the amount of infrastructure and platform responsibility retained by the customer. A managed Kubernetes platform with a cloud load balancer and managed database is a suitable design for the result portal. Pods run containers; Deployments maintain desired replicas; Services provide stable routing; requests and limits guide scheduling and constrain resource use; HPA adjusts Pod count using metrics. A managed node autoscaler supplies additional worker nodes when needed. Fixed result-day VMs provide immediate capacity but leave substantial resources unused on ordinary days. The recommended approach is managed Kubernetes with planned pre-scaling, explicit limits and a separately protected database.

8. Technical References

[1] NIST, *SP 800-145: The NIST Definition of Cloud Computing* - IaaS, PaaS and SaaS service models. https://csrc.nist.gov/pubs/sp/800/145/final

[2] Kubernetes Documentation, *Pods* - smallest deployable unit and shared resources. https://kubernetes.io/docs/concepts/workloads/pods/

[3] Kubernetes Documentation, *Deployments* - desired replicas, ReplicaSets and rolling updates. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[4] Kubernetes Documentation, *Services, Load Balancing, and Networking* - stable service endpoints and LoadBalancer integration. https://kubernetes.io/docs/concepts/services-networking/

[5] Kubernetes Documentation, *Resource Management for Pods and Containers* - CPU/memory requests and limits. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[6] Kubernetes Documentation, *Horizontal Pod Autoscaling* - metrics, CPU requests and desired replica calculation. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Source alignment: Assignment 5 - CO5, Part A - Design, Task A5 - Design the Cloud Deployment and Orchestration, 4 Marks, page 45 of the supplied question paper. General assignment instructions appear on page 42.

[7] Kubernetes Documentation, Node Autoscaling - provisioning and consolidation of worker nodes alongside horizontal workload autoscaling. https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/